ECA5801-IOE AND AI DRIVEN SYSTEM

SIMULATION

EXPERIMENT 9

K.POOJA SREE

192512245

Simulation of Telemetry Data Ingestion to Time-Series Database and Visualization using Grafana, Power BI and Tableau

```
# ============================================================
# EXPERIMENT: Simulation of Telemetry Data Ingestion to Time-Series
# Database and Visualization using Grafana, Power BI and Tableau
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timezone
from IPython.display import display
from IPython.display import Markdown

# ---------- 1. PARAMETERS ----------

NUM_DEVICES = 5
INTERVAL_SECONDS = 1
DURATION_MINUTES = 30
REFRESH_SECONDS = 5
AGGREGATION_MINUTES = 1
```

```
RETENTION_DAYS = 7


EXPECTED_RECORDS = (
    NUM_DEVICES * DURATION_MINUTES * 60 // INTERVAL_SECONDS
)


np.random.seed(42)


print("=" * 65)
print("IoE TELEMETRY SIMULATION AND PERFORMANCE EVALUATION")
print("=" * 65)
print(f"Number of devices        : {NUM_DEVICES}")
print(f"Publishing interval      : {INTERVAL_SECONDS} second")
print(f"Simulation duration      : {DURATION_MINUTES} minutes")
print(f"Expected records         : {EXPECTED_RECORDS}")
print(f"MQTT topic pattern       : ieo/telemetry/<device_id>")
print(f"QoS setting in design    : 1")
print(f"Database bucket in design: telemetry")
print(f"Retention period         : {RETENTION_DAYS} days")
print(f"Dashboard refresh target : {REFRESH_SECONDS} seconds")
print(f"Aggregation window       : {AGGREGATION_MINUTES} minute")
print("=" * 65)


# ---------- 2. TELEMETRY GENERATION ----------


start_time = datetime.now(timezone.utc).replace(microsecond=0)
```

```
timestamps = pd.date_range(
    start=start_time,
    periods=DURATION_MINUTES * 60,
    freq="s"
)

rows = []
injected_spikes = [300, 600, 900, 1200, 1500]

for device_num in range(1, NUM_DEVICES + 1):

    device_id = f"device_{device_num}"
    n = len(timestamps)

    temperature = 25 + np.random.normal(0, 0.8, n)
    humidity = 55 + np.random.normal(0, 4, n)
    vibration = np.abs(np.random.normal(2, 0.5, n))
    battery = np.clip(
        100 - np.arange(n) * 0.005
        + np.random.normal(0, 0.15, n),
        0, 100
    )

    # Inject five temperature spikes into device 1
    if device_num == 1:
```

```
        for idx in injected_spikes:
            temperature[idx] += 15

    for i, ts in enumerate(timestamps):
        rows.append({
            "timestamp": ts,
            "device_id": device_id,
            "temperature": round(float(temperature[i]), 2),
            "humidity": round(float(np.clip(humidity[i], 0, 100)), 2),
            "vibration": round(float(vibration[i]), 2),
            "battery": round(float(battery[i]), 2)
        })

df = pd.DataFrame(rows).sort_values(
    ["timestamp", "device_id"]
).reset_index(drop=True)

print("\n1. TELEMETRY GENERATION")
print("-" * 45)
print(f"Generated records : {len(df)}")
print(f"Unique devices    : {df['device_id'].nunique()}")
print(f"UTC timestamps    : {df['timestamp'].dt.tz is not None}")
display(df.head(10))

# ---------- 3. INGESTION EVALUATION ----------
```

```
# Demonstration assumption: all generated records are stored.
# Replace stored_records with actual database count when available.
published_records = len(df)
stored_records = published_records

data_loss = (
    (published_records - stored_records) / published_records * 100
    if published_records else 0
)

duration_seconds = DURATION_MINUTES * 60
throughput = stored_records / duration_seconds

percentage_deviation = (
    abs(stored_records - EXPECTED_RECORDS) / EXPECTED_RECORDS * 100
    if EXPECTED_RECORDS else 0
)

# Illustrative latency only; not measured from a real network.
simulated_latency = 0.5

print("\n2. INGESTION PERFORMANCE")
print("-" * 45)

performance = pd.DataFrame({
    "Output Parameter": [
```

```
        "Expected records",
        "Generated records",
        "Stored records (assumed)",
        "Data loss (%)",
        "Throughput (records/second)",
        "Illustrative latency (seconds)",
        "Percentage deviation (%)",
        "Data loss below 1%",
        "Latency below 2 seconds"
    ],
    "Value": [
        EXPECTED_RECORDS,
        published_records,
        stored_records,
        round(data_loss, 2),
        round(throughput, 2),
        simulated_latency,
        round(percentage_deviation, 2),
        data_loss < 1,
        simulated_latency < 2
    ]
})

display(performance)

# ---------- 4. ANOMALY DETECTION ----------
```

```
mean_temp = df["temperature"].mean()
std_temp = df["temperature"].std()

df["anomaly"] = (
    (df["temperature"] - mean_temp).abs() > 3 * std_temp
)

anomalies = df[df["anomaly"]].copy()

print("\n3. THREE-SIGMA ANOMALY DETECTION")
print("-" * 45)
print(f"Overall mean temperature : {mean_temp:.2f} °C")
print(f"Overall standard deviation: {std_temp:.2f} °C")
print(f"Detected anomaly records : {len(anomalies)}")
print("Note: detected count depends on the generated data and threshold.")
display(
    anomalies[
        ["timestamp", "device_id", "temperature", "anomaly"]
    ].head(20)
)

# ---------- 5. TEMPERATURE TIME SERIES ----------

plt.figure(figsize=(13, 5))
```

```
for device_id, group in df.groupby("device_id"):
    plt.plot(
        group["timestamp"],
        group["temperature"],
        label=device_id,
        linewidth=1
    )

if not anomalies.empty:
    plt.scatter(
        anomalies["timestamp"],
        anomalies["temperature"],
        marker="x",
        s=55,
        label="Detected anomalies"
    )

plt.title("Temperature Telemetry by Device")
plt.xlabel("Timestamp (UTC)")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(True)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

```
# ---------- 6. BATTERY GAUGE-STYLE PLOT ----------


latest = (
    df.sort_values("timestamp")
      .groupby("device_id")
      .tail(1)
      .sort_values("device_id")
)


plt.figure(figsize=(9, 4))
plt.bar(latest["device_id"], latest["battery"])
plt.axhline(
    20,
    linestyle="--",
    label="Low battery threshold (20%)"
)
plt.ylim(0, 110)
plt.title("Latest Battery Level by Device")
plt.xlabel("Device")
plt.ylabel("Battery (%)")
plt.legend()
plt.grid(axis="y")
plt.tight_layout()
plt.show()


# ---------- 7. LATEST HUMIDITY STAT ----------
```

```
latest_humidity = df.iloc[-1]["humidity"]


print("\n4. LATEST HUMIDITY STAT")

print("-" * 45)

print(f"Latest humidity reading: {latest_humidity:.2f}%")


# ---------- 8. LATEST TELEMETRY TABLE ----------


print("\n5. LATEST TELEMETRY TABLE")

print("-" * 45)


display(

    latest[

        [

            "timestamp",

            "device_id",

            "temperature",

            "humidity",

            "vibration",

            "battery"

        ]

    ].reset_index(drop=True)

)


# ---------- 9. ONE-MINUTE AGGREGATION ----------
```

```
df["minute"] = df["timestamp"].dt.floor("min")


aggregated = (
    df.groupby(["minute", "device_id"], as_index=False)
      .agg(
          mean_temperature=("temperature", "mean"),
          mean_humidity=("humidity", "mean"),
          mean_vibration=("vibration", "mean"),
          mean_battery=("battery", "mean")
      )
)

print("\n6. ONE-MINUTE MEAN AGGREGATION")
print("-" * 45)
display(aggregated.head(15))


plt.figure(figsize=(12, 5))


for device_id, group in aggregated.groupby("device_id"):
    plt.plot(
        group["minute"],
        group["mean_temperature"],
        marker="o",
        label=device_id
    )
```

```
plt.title("One-Minute Mean Temperature by Device")
plt.xlabel("Timestamp (UTC)")
plt.ylabel("Mean Temperature (°C)")
plt.legend()
plt.grid(True)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# ---------- 10. CSV EXPORT ----------

df.to_csv("ioe_telemetry_data.csv", index=False)
aggregated.to_csv("ioe_telemetry_1min_mean.csv", index=False)

print("\n7. CSV EXPORT")
print("-" * 45)
print("Created: ioe_telemetry_data.csv")
print("Created: ioe_telemetry_1min_mean.csv")

# ---------- 11. TOOL COMPARISON ----------

comparison = pd.DataFrame({
    "Criterion": [
        "Data connection to time-series DB",
        "Real-time refresh",
```

```
    "Dashboard build time (minutes)",
    "Aggregation and filtering",
    "Alerting support",
    "Best suited for"
],
"Grafana": [
    "Native InfluxDB integration",
    "Strong; configurable auto-refresh",
    "Measure experimentally",
    "Time-series queries and aggregation",
    "Built-in alerting options",
    "Real-time IoE monitoring"
],
"Power BI": [
    "Connector/API or supported DB connection",
    "Depends on connection and refresh mode",
    "Measure experimentally",
    "Analytics, visuals and slicers",
    "Depends on product features and setup",
    "Historical analytics and reporting"
],
"Tableau": [
    "Connector/API or supported DB connection",
    "Depends on live/extract connection",
    "Measure experimentally",
    "Interactive filtering and analytics",
```

```
        "Depends on product features and setup",
        "Interactive data exploration"
    ]
})

print("\n8. COMPARATIVE VISUALIZATION EVALUATION")
print("-" * 45)
display(comparison)

# -------------------- 12. FINAL SUMMARY --------------------------

print("\n" + "=" * 65)
print("EXPERIMENT SUMMARY")
print("=" * 65)
print(f"Expected records            : {EXPECTED_RECORDS}")
print(f"Generated records           : {len(df)}")
print(f"Assumed stored records      : {stored_records}")
print(f"Assumed data loss           : {data_loss:.2f}%")
print(f"Calculated throughput       : {throughput:.2f} records/s")
print(f"Illustrative latency        : {simulated_latency:.2f} seconds")
print(f"Detected temperature anomalies: {len(anomalies)}")
print(f"Aggregated rows             : {len(aggregated)}")
print("CSV files                   : 2 created")
print("=" * 65)

print("\nRESULT:")
```

```
print(
    "Telemetry data was simulated, analyzed, aggregated, "
    "visualized, and exported for use in Power BI and Tableau. "
    "Grafana-style monitoring outputs were represented using "
    "Python plots. Actual MQTT/database ingestion and dashboard "
    "refresh performance require testing the real services."
)


# Automatically download both CSV files in Google Colab
try:
    from google.colab import files
    files.download("ioe_telemetry_data.csv")
    files.download("ioe_telemetry_1min_mean.csv")
except ImportError:
    print("CSV files are saved in the current working directory.")
```

OUTPUT:

```
================================================================
IoE TELEMETRY SIMULATION AND PERFORMANCE EVALUATION
================================================================
Number of devices        : 5
Publishing interval      : 1 second
Simulation duration      : 30 minutes
Expected records         : 9000
MQTT topic pattern       : ieo/telemetry/<device_id>
QoS setting in design    : 1
Database bucket in design: telemetry
Retention period         : 7 days
Dashboard refresh target : 5 seconds
Aggregation window       : 1 minute
================================================================

1. TELEMETRY GENERATION
------------------------------------------------
Generated records : 9000
Unique devices    : 5
UTC timestamps    : True
```

	timestamp	device_id	temperature	humidity	vibration	battery
0	2026-10-28 13:36:33+00:00	device_1	25.40	62.60	1.59	99.82
1	2026-10-28 13:36:33+00:00	device_2	25.11	58.06	1.72	100.00
2	2026-10-28 13:36:33+00:00	device_3	24.96	60.60	2.16	100.00
3	2026-10-28 13:36:33+00:00	device_4	25.32	56.35	1.84	99.88
4	2026-10-28 13:36:33+00:00	device_5	24.03	51.23	2.36	99.71
5	2026-10-28 13:36:34+00:00	device_1	24.89	54.76	1.64	99.94
6	2026-10-28 13:36:34+00:00	device_2	25.05	59.29	2.26	99.98
7	2026-10-28 13:36:34+00:00	device_3	23.96	56.82	1.61	100.00
8	2026-10-28 13:36:34+00:00	device_4	24.34	52.41	1.48	100.00
9	2026-10-28 13:36:34+00:00	device_5	24.14	57.67	1.69	100.00

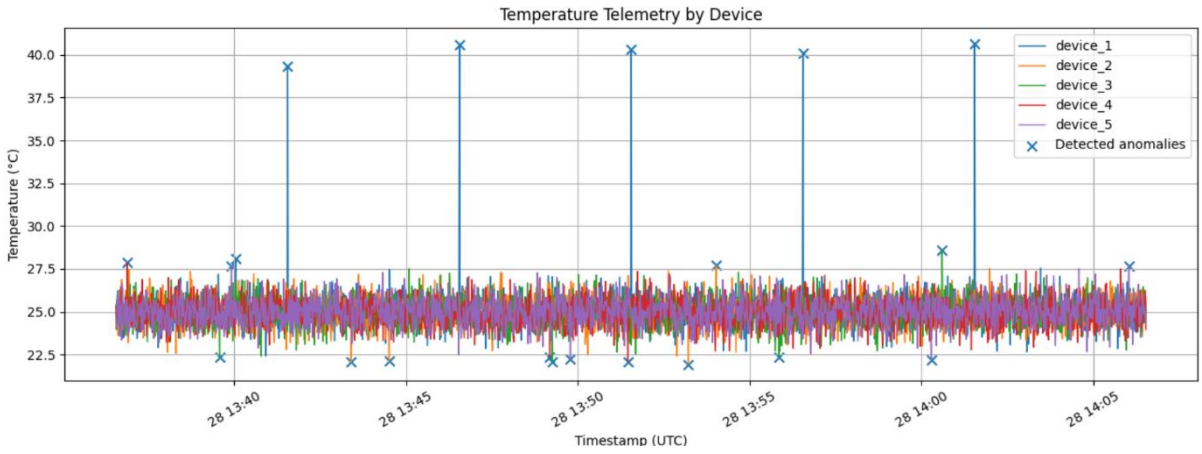
2. INGESTION PERFORMANCE

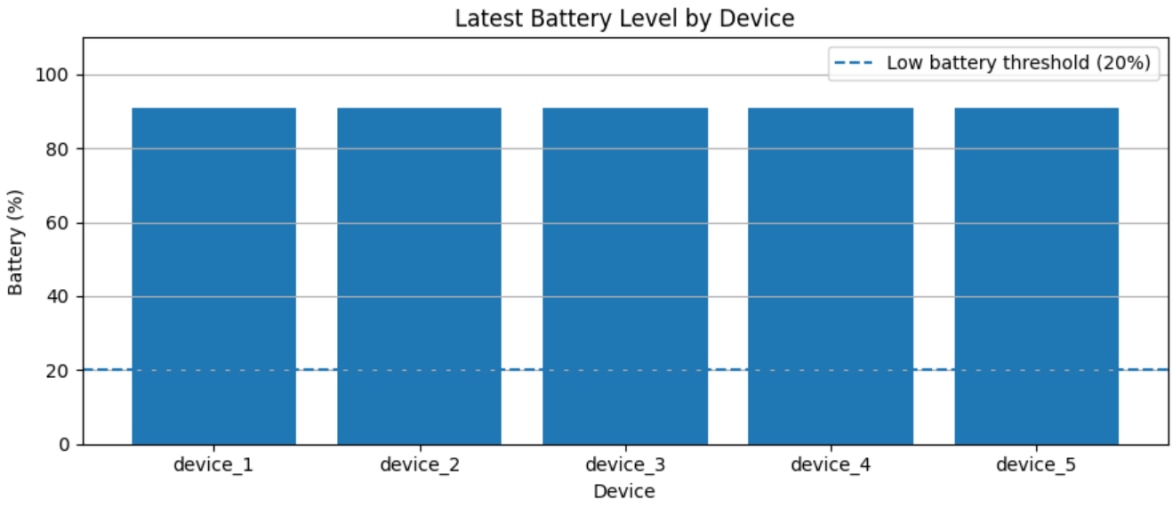
	Output Parameter	Value
0	Expected records	9000
1	Generated records	9000
2	Stored records (assumed)	9000
3	Data loss (%)	0.0
4	Throughput (records/second)	5.0
5	Illustrative latency (seconds)	0.5
6	Percentage deviation (%)	0.0
7	Data loss below 1%	True
8	Latency below 2 seconds	True

3. THREE-SIGMA ANOMALY DETECTION

Overall mean temperature : 25.02 °C
Overall standard deviation: 0.88 °C
Detected anomaly records : 21
Note: detected count depends on the generated data and threshold.

	timestamp	device_id	temperature	anomaly
103	2026-10-28 13:36:53+00:00	device_4	27.89	True
907	2026-10-28 13:39:34+00:00	device_3	22.34	True
1014	2026-10-28 13:39:55+00:00	device_5	27.68	True
1045	2026-10-28 13:40:02+00:00	device_1	28.08	True
1500	2026-10-28 13:41:33+00:00	device_1	39.34	True
2056	2026-10-28 13:43:24+00:00	device_2	22.05	True
2386	2026-10-28 13:44:30+00:00	device_2	22.12	True
3000	2026-10-28 13:46:33+00:00	device_1	40.61	True
3793	2026-10-28 13:49:11+00:00	device_4	22.36	True
3812	2026-10-28 13:49:15+00:00	device_3	22.09	True
3974	2026-10-28 13:49:47+00:00	device_5	22.26	True
4473	2026-10-28 13:51:27+00:00	device_4	22.08	True
4500	2026-10-28 13:51:33+00:00	device_1	40.29	True
5001	2026-10-28 13:53:13+00:00	device_2	21.93	True
5241	2026-10-28 13:54:01+00:00	device_2	27.70	True
5792	2026-10-28 13:55:51+00:00	device_3	22.35	True
6000	2026-10-28 13:56:33+00:00	device_1	40.10	True
7124	2026-10-28 14:00:17+00:00	device_5	22.17	True
7217	2026-10-28 14:00:36+00:00	device_3	28.58	True
7500	2026-10-28 14:01:33+00:00	device_1	40.62	True





4. LATEST HUMIDITY STAT

Latest humidity reading: 52.82%

5. LATEST TELEMETRY TABLE

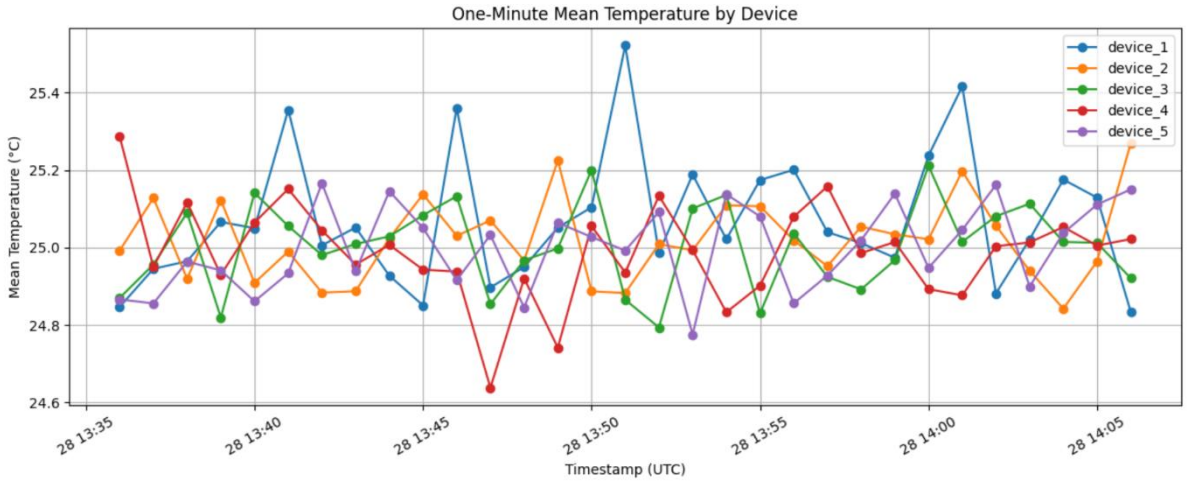
---

	timestamp	device_id	temperature	humidity	vibration	battery
0	2026-10-28 14:06:32+00:00	device_1	25.02	54.09	2.16	90.78
1	2026-10-28 14:06:32+00:00	device_2	25.64	59.89	2.22	91.04
2	2026-10-28 14:06:32+00:00	device_3	25.14	59.13	1.98	90.81
3	2026-10-28 14:06:32+00:00	device_4	23.96	54.13	1.70	91.09
4	2026-10-28 14:06:32+00:00	device_5	24.35	52.82	2.18	91.02

6. ONE-MINUTE MEAN AGGREGATION

---

	minute	device_id	mean_temperature	mean_humidity	mean_vibration	mean_battery
0	2026-10-28 13:36:00+00:00	device_1	24.847037	53.794074	1.978519	99.888889
1	2026-10-28 13:36:00+00:00	device_2	24.992593	54.522222	1.937407	99.915926
2	2026-10-28 13:36:00+00:00	device_3	24.870370	56.693704	2.014815	99.934444
3	2026-10-28 13:36:00+00:00	device_4	25.286667	55.295185	1.958148	99.928889
4	2026-10-28 13:36:00+00:00	device_5	24.866296	54.437037	2.097407	99.908148
5	2026-10-28 13:37:00+00:00	device_1	24.945333	55.143000	2.037500	99.711500
6	2026-10-28 13:37:00+00:00	device_2	25.128167	55.210500	1.953500	99.727833
7	2026-10-28 13:37:00+00:00	device_3	24.956500	54.795833	1.958667	99.721000
8	2026-10-28 13:37:00+00:00	device_4	24.950000	56.602667	2.062833	99.678167
9	2026-10-28 13:37:00+00:00	device_5	24.856000	55.483833	2.037667	99.670833
10	2026-10-28 13:38:00+00:00	device_1	24.964833	55.122833	2.004167	99.419833
11	2026-10-28 13:38:00+00:00	device_2	24.919500	55.777833	1.878333	99.435500




```
7. CSV EXPORT
------------------------------------------------
Created: ioe_telemetry_data.csv
Created: ioe_telemetry_1min_mean.csv

8. COMPARATIVE VISUALIZATION EVALUATION
------------------------------------------------
```

	Criterion	Grafana	Power BI	Tableau
0	Data connection to time-series DB	Native InfluxDB integration	Connector/API or supported DB connection	Connector/API or supported DB connection
1	Real-time refresh	Strong; configurable auto-refresh	Depends on connection and refresh mode	Depends on live/extract connection
2	Dashboard build time (minutes)	Measure experimentally	Measure experimentally	Measure experimentally
3	Aggregation and filtering	Time-series queries and aggregation	Analytics, visuals and slicers	Interactive filtering and analytics
4	Alerting support	Built-in alerting options	Depends on product features and setup	Depends on product features and setup
5	Best suited for	Real-time IoE monitoring	Historical analytics and reporting	Interactive data exploration

```
================================================================
EXPERIMENT SUMMARY
================================================================
Expected records             : 9000
Generated records            : 9000
Assumed stored records        : 9000
Assumed data loss             : 0.00%
Calculated throughput         : 5.00 records/s
Illustrative latency          : 0.50 seconds
Detected temperature anomalies: 21
Aggregated rows               : 155
CSV files                     : 2 created
================================================================
```